Improving Context-Aware Music Recommender Systems with a Dual Recurrent Neural Network

Igor André Pegoraro Santana and Marcos Aurélio Domingues

State University of Maringá
Department of Informatics
Maringá, Brazil
{pg400816, madomingues}@uem.br

Abstract. Day by day, online content delivery services suppliers grow the volume of data on the internet. Music streaming services are one of those services that increase the number of users every day, as well as the number of songs in their catalog. To help the users to find songs that fit their interests, music recommender systems can be used to filter a large number of songs according to the preference of the user. However, the context in which the users listen to songs must be taken into account, which justifies the usage of context-aware recommender systems. The goal of this work is to use a Dual Recurrent Neural Network to acquire contextual information (represented by embeddings) for each song, given the sequence of songs that each user has listened to. We evaluated the embeddings by using four context-aware music recommender systems in two datasets. The results showed that the embeddings (i.e. the contextual information) obtained by our proposed method presents better results than the state-of-the-art method proposed in the literature.

Keywords: Recurrent Neural Networks · Context-Aware Recommender Systems · Music Recommendation · Embeddings · Context Acquisition.

1 Introduction

With the growing of music streaming services nowadays, the abundance of available songs for the users grows as well. Spotify[1], as an example, has 50 million songs available in its directory. Users can not handle so much data, making it necessary for the system to implement a tool that assists users in finding songs that are fit for their preferences.

The usage of smartphones with music streaming services changed how people listen to music. A user can be texting a friend, browsing through its social networks or answering e-mails, while listening to music in the background. In this way, a user is inserted in a broader context while listening to a song. Also, as seen in [12], people look for songs based on occasions, events and emotions, which suggests that listening to songs is not an isolated event.

[1] https://www.spotify.com

A useful tool to deal with this information overload is a recommender system, which can recommend songs to users based on their preferences. Several works propose and review music recommender systems [4], [2], [3],[13], however, knowing that users usually listen to songs given a context, we can replace it by context-aware recommender systems, which can include this kind of information in their model. The work of [11] reviews some context-aware music recommender systems.

Although there are some works about context-aware music recommender systems, there is a lack of automatic techniques for extracting contextual information. To obtain such information, we proposed a Dual Recurrent Neural Network model that uses Long Short-Term Memory (LSTM) networks to analyze the sequence of songs that the users listened to and to produce an embedding vector (i.e. a contextual information) for each song. Embedding is a method to represent items using a real values vector that is obtained by using a neural network trained to understand the context of the items. LSTM is a kind of Recurrent Neural Network (RNN) that was proposed with the intent to solve problems with long term dependencies [9]. The embedding vector contains the contextual information that encircle the songs for each user.

We evaluated our proposal by using four context-aware recommender systems in two music datasets that include the listening history of thousands of users. As a baseline, to compare our proposal against to, we used the model proposed by [21]. That model was based on the Skip-Gram architecture [15], a state-of-the-art embedding model. The results showed that, in both datasets, our proposed model outperformed the baseline, indicating that it can capture better contextual information through the embedding vector.

The remaining of this paper is organized as follows: In Section 2, we describe some works that use embedding vectors and context-aware music recommender systems. Section 3 contains our proposed model. Section 4 describes the empirical evaluation, i.e. the datasets, the recommender systems, the evaluation setup, and the results. Finally, in Section 5, we present our conclusions and future directions.

2 Related Work

Context-aware music recommender systems have gained notoriety in recent years, with several researches being published in this area. To the best of our knowledge, most of the works [22], [15], [21], [14] that tried to recommend songs to a user, using contextual information, obtained such information by analyzing the sessions from the user, which are the sequence of songs that the user has listened to. Only one work did not use sessions to infer the contextual information [7]. That previous work uses emotions as contextual information, in which the emotions obtained from the user's tweets were encoded in fixed-sized tweets, using an adaptation of the bag-of-words method.

A common and effective approach to extract contextual information from sequences of data is the Skip-Gram model [15]. In [22], the authors included additional information about songs (metadata) in the Skip-Gram model. In [21], the

authors proposed a Skip-Gram based model that produces embeddings based on the general preferences from the user, and embeddings based on the preferences that come from the sessions (i.e. contextual preferences).

In [20], an AutoEncoder model was proposed to obtain embeddings to the next song recommendation task. The work does not only encode the songs using the AutoEncoder, but also encode the playlists to estimate the most similar songs to the playlists.

3 Proposed Work

Inspired by the word2vec model proposed in [15], Wang et al. introduced a model to obtain the embedding vectors from songs [21]. The model is based on the Skip-Gram architecture and consists of two methods: music2vec and session-music2vec. The methods obtain embeddings from songs with different goals: one goal was to obtain the general preference from the user (music2vec) and another one was to obtain the contextual preference from the user (session-music2vec). The user's general preference can be inferred by its complete listening history and refers to the user's specific preferences for music. The contextual preference for songs indicates the recent preferences of the user in the current session. Figure 1 illustrates the model, which the main idea is that the sequence of songs listened by a user reflects its song preferences during that period, and that co-occurrence of songs in a sequence indicates that those songs are similar. Embeddings of songs that are close in a sequence must appear close in a low dimensional space. Here, we used the work proposed in [21] as the baseline.

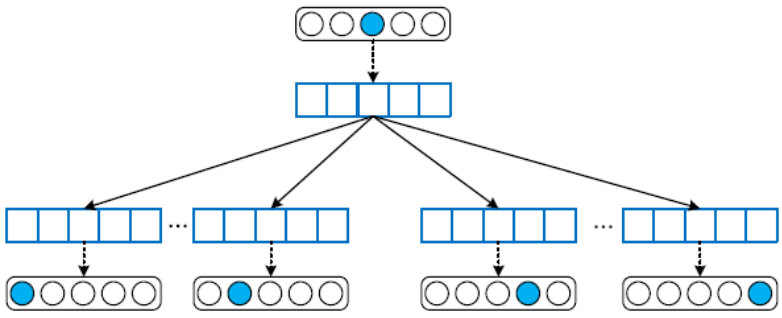

Fig. 1. The skip-gram architecture for music2vec and session-music2vec.

As already stated, embedding is a method to represent items using a real values vector that is obtained by using a neural network trained to understand the context of the items. Instead of using traditional neural networks to obtain the embeddings, as proposed in [21], our model captures the context of the songs by using a Recurrent Neural Network (RNN).

In our work, we propose a Dual Recurrent Neural Network that is able to learn the general and contextual preferences in a same model, generating embeddings (i.e. contextual information) for the songs that can be used in context-aware music recommenders. Similar to the Context Bag-of-Words model (CBOW) proposed by [15], the goal of our model is to predict the center song in the contextual window given its neighborhood songs. Figure 2 contains an overview of the proposed model and its most important layers. However, in contrast to the CBOW model, as we use a Recurrent Neural Network (i.e. LSTM layers) to analyze the contextual windows, the order in which the songs are in the window matter.

Long Short-Term Memory (LSTM) is a kind of Recurrent Neural Networks that was proposed with the intent to solve problems with long term dependencies [9]. It uses gated cells that are capable to forget information that will no longer be useful, and keep information that can be used later on the sequence [8]. There are three gates in the LSTM cell: forget gate, input gate, and output gate. Through those gates, the LSTM cell learns which information is useful in a sequence and passes that information to make predictions through the output gate, and the cell state containing the relevant information is passed to the next timestamp.

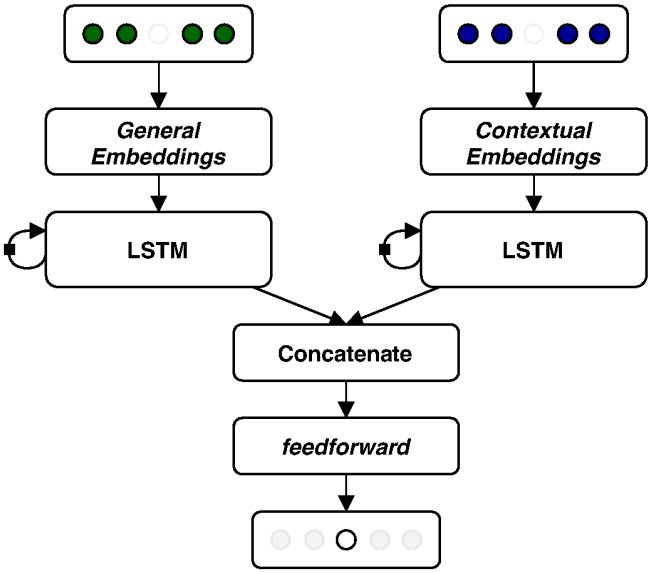

Fig. 2. Overview of the proposed model.

As defined by [21], let $U = \{u_1, u_2, \ldots, u_{|U|}\}$ be the set of users and $M = \{m_1, m_2, \ldots, m_{|M|}\}$ be the set of songs, in which $|U|$ e $|M|$ are the total number of unique users and songs, respectively. For each user u, their listening history

are the songs that were listened to by the user with its respective date and time, defined as $H^u = \left\{m_1^u, m_2^u, \ldots, m_{|H^u|}^u\right\}$.

According to how much time has passed between two songs, the user's listening history can be divided into sessions $S^u = \left\{S_1^u, S_2^u, \ldots, S_{|S^u|}^u\right\}$. A session n from user u is defined as $S_n^u = \left\{m_{n,1}^u, m_{n,2}^u, \ldots, m_{n,|s'_n|}^u\right\}$, in which $m_{n,j}^u \in M$.

As seen in Figure 2, our model receives streams of data: the left one, which has as input the sliding window of a song m_i^u, taking into the complete listening history of a user H^u, for all users. The right stream has as input the sliding window of the same song $m_{n,i}^u$, taking into account the session which the song is, instead of the whole listening history.

Those sliding windows are passed to the LSTM layers, that are responsible to analyze the hidden relationships between the sequences of songs in the windows of both streams of data. Then, the cell state of the last timestamp of both LSTMs are concatened in a single vector. Finally, this vector is used as input by a fully connected feedforward layer, that tries to predict the song that is in the center of both sliding windows, using as input the concatenated vector.

Each part of the model has its own embedding matrix that is initialized with all the songs in the dataset and, as the model learns through the forward and back-propagation process, this matrix gets updated. These embeddings will later be used in the context-aware recommender systems. The result of this training process is that each song will have two embedding vectors: one that refers to the general preferences from the users and another one that refers to the contextual preferences from the users. Thus, we can explore the flexibility of neural networks in a single model to learn the two embedding vectors, instead of two models as proposed in [21].

The implementation of our proposed model is available in the GitHub[2].

4 Empirical Evaluation

In this section, we describe the empirical evaluation carried out to evaluate our proposed model. In Subsection 4.1, we present the datasets and their main statistics. Subsection 4.2 introduces the context-aware recommender systems proposed by [21] that were used to evaluate our model against the baseline. In Subsection 4.3, we present the evaluation setup. The results are discussed in Subsection 4.4.

4.1 Datasets

The two datasets used in the empirical evaluation contain the listening history for each user as well as the timestamp for each listening event, without any information about sessions. So, to generate the sessions, we decided to split songs that were listened 30 minutes apart from each other. The first dataset

[2] `https://github.com/igorsantana/rnn-embeddings`

was proposed by [21] and was built using a web crawler on the application Xiami Music[3], referred here as Xiami[4]. The dataset has 361,899 songs; and 4,284 users with 1,000 listened songs in its listening history, with an average of 370 unique songs per user. The second dataset, called Music4All[5] [18], was created by obtaining the listening history of random users from the last.fm[6] application through its official API. The dataset has many more users (15,602), but there are fewer songs per user, with an average of 361 songs per user, and 184 unique songs per user. The total number of song in this dataset is 109,269.

4.2 Context-Aware Recommender Systems

To evaluate the embeddings (i.e. contextual information) from our proposed model against the ones from the baseline, we used the four context-aware recommender systems proposed by [21]. The recommenders make use of a general preference and a contextual preference for each user, which are built based on the learned embeddings. The general preference for a user u can be learned from its entire listening history $H^u = \left\{m_1^u, m_2^u, \ldots, m_{|H^u|}^u\right\}$ and is defined as:

$$\mathbf{p}_g^u = \frac{1}{|H^u|} \sum_{m_i^u \in H^u} \mathbf{v}_{m_i^u}^{g2v}, \tag{1}$$

where $\mathbf{v}_{m_i^u}^{g2v}$ is defined as the general embedding vector. The contextual preference for the user u, given their current session $S_n^u = \left\{m_{n,1}^u, m_{n,2}^u, \ldots, m_{n,|S_i^u|}^u\right\}$ can be defined as:

$$\mathbf{p}_c^u = \frac{1}{|S_n^u|} \sum_{m_{n,i}^u \in S_n^u} \mathbf{v}_{m_{n,i}^u}^{c2v}, \tag{2}$$

where $\mathbf{v}_{m_{n,i}^u}^{c2v}$ corresponds to the contextual embedding vector for the song. As we can see in Equation 1, the general preference is defined as an average of all the general embedding vectors of the songs in the user's listening history. On the other hand, the contextual preference, defined in Equation 2, is the average of all the contextual embedding vectors of the songs in the user's current session. Given the general and contextual embedding vectors as well as the preferences for each user, four context-aware recommender systems were defined in [21]: Music2vec-TopN (M-TN), Session-Music2vec-TopN (SM-TN), Context-Session-Music2vec-TopN (CSM-TN) and Context-Session-Music2vec-UserKNN (CSM-UK).

Of all the context-aware recommender systems, the M-TN is the only one that uses only the general preference (i.e. the general embedding vector) to

[3] https://www.xiami.com
[4] https://1drv.ms/f/s!ApojZBGe9UzXgaI6x8pBf8JgN4PfZg
[5] https://sites.google.com/view/contact4music4all
[6] https://www.last.fm

recommend songs to the users. Given a user u and their general preference $\mathbf{p}_g^u$ for songs, the recommender system measures the cosine similarity between $\mathbf{p}_g^u$ and the general embedding vector of all the songs in the set of songs M. The top-N songs with the highest value of cosine similarity are recommended to the user. Formally, the predicted preference of the user u to the song m can be defined as:

$$pp_{M-TN}(u, m) = \cos\left(\mathbf{p}_g^u, \mathbf{v}_m^{g2v}\right). \tag{3}$$

The SM-TN recommender system is similar to M-TN, but it uses contextual information instead of the general information. Given a user u and their contextual preference $\mathbf{p}_c^u$, the SM-TN measures the cosine similarity between the contextual embedding vector $\mathbf{v}_{m_{n,i}^u}^{c2v}$ of the songs and the contextual preference of the user. The top-N songs with the highest cosine similarity are then recommended to the user. Formally, the preference can be defined as:

$$pp_{SM-TN}(u, m) = \cos\left(\mathbf{p}_c^u, \mathbf{v}_m^{c2v}\right). \tag{4}$$

The CSM-TN recommender system is a combination of the previous recommender systems: M-TN and SM-TN. After the similarity of each recommender is calculated for each song, they are summed to obtain the most similar songs according to both the contextual and general preferences of the user. Formally, the preference is defined as:

$$\mathrm{PP}_{CSM-TN}(u, m) = \cos\left(\mathbf{p}_g^u, \mathbf{v}_m^{g2v}\right) + \cos\left(\mathbf{p}_c^u, \mathbf{v}_m^{c2v}\right). \tag{5}$$

The last recommender system, CSM-UK, proposes a combination of the traditional recommender system, UserKNN [17], with the learned embedding vectors. The UserKNN recommender system needs a similarity function to build a neighborhood of similar users. In [21], the similarity function between two users, u and v is defined as follows:

$$\mathrm{sim}(u, v) = \sum_{m \in M_u \cap M_v} \frac{1}{\sqrt{|M_u| \times |M_v|} + \cos\left(\mathbf{p}_g^u, \mathbf{p}_g^v\right)}, \tag{6}$$

where M_u e M_v are the set of songs listened by the users u and v, respectively. With the similarity function, the CSM-UK system recommends the top-N most similar songs for each user, given the user contextual preference and their most similar users. The predicted preference for the target user u to a song m can be defined as:

$$pp_{CSM-UK}(u, m) = \left(\sum_{v \in U_{u,K} \cap U_m} \frac{\mathrm{sim}(u, v)}{|U_{u,k} \cap U_m|} \right) + \cos\left(\mathbf{p}_c^u, \mathbf{v}_m^{c2v}\right), \tag{7}$$

where $U_{u,K}$ is the set with the K users more similar to u, and U_m is the set of users who have listened to song m.

4.3 Evaluation Setup

To verify if our model is able of achieving better results than the baseline, the context-aware recommender systems were executed using the k-fold cross-validation protocol. In this protocol, we split the users of the datasets into k mutually exclusive partitions, in which 1 of these partitions is chosen as the testing partition and the remaining are chosen as the training partition. We chose the testing partition k times, without repeating the same partition, as seen in [1]. Figure 3 shows the process of splitting the users into partitions, assuming $k = 5$ that is the value used in our work.

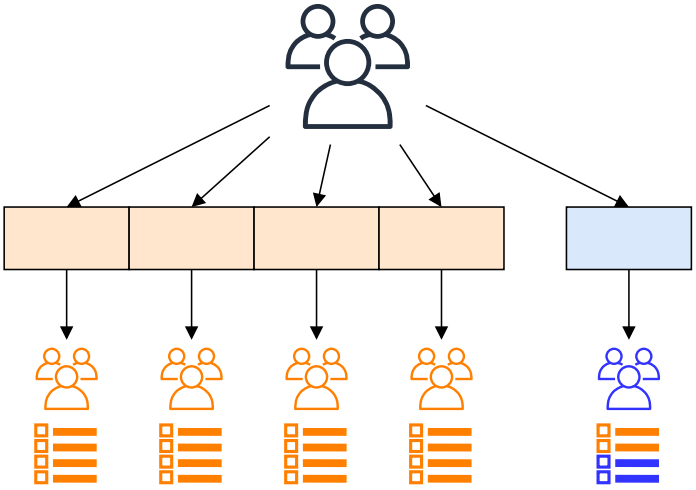

Fig. 3. The k-fold cross-validation protocol with $k = 5$.

Users that are in the training partition use all of their songs sessions to build their preferences (general and contextual), as it can be seen in Figure 3. As for the users that are in the testing partitions, they use only the first part of their sessions to build their preferences, and the second part for each session is used as the testing songs.

To evaluate the recommendations made by the recommender systems, we used five different metrics, which three (Precision, Recall, and F-measure) are commonly used to evaluate the accuracy of the recommendations, and two metrics that are used to evaluate if the ranking of the recommendations meets the user's preference, which are Mean Average Precision (MAP) and Normalized Discounted Cumulative Gain (NDCG) [10].

Precision, as described by [6], is a metric that measures the proportion of satisfying recommendations made by the recommender system, indicating the quality of recommendations made with an emphasis on the success of the recommendations. As seen in [19], Precision can be defined as:

$$Precision = \frac{|tp|}{|tp| + |fp|}, \tag{8}$$

where, according to Table 1, tp means true positive and fp means false positive.

Recall, on the other hand, measures the proportion of the recommendations among the songs that the user is actually interested in. [19] defined Recall as:

$$Recall = \frac{|tp|}{|tp| + |fn|}, \tag{9}$$

where, according to Table 1, tp means true positive and fn means false negative.

The F-measure metric is defined as the harmonic mean between the Precision and Recall metrics and as those metrics, its value varies between 0 and 1. As seen in [19], F-measure can be defined as:

$$F\text{-}measure = 2 * \frac{Precision * Recall}{Precision + Recall}. \tag{10}$$

The ranking metrics that were used in this work have different goals. MAP, for instance, as described in [19], has as its main focus to ensure that the first few items in the recommendation list are in the correct order. If they are not, the metric will penalize the recommendation list.

The MAP metric can be calculated as a mean of the Average Precision (AP) metric for each recommendation made for a single user. As defined by [19], the metric AP can be calculated for a recommendation list with N items (in our case, songs) as:

$$AP@N = \frac{1}{N} \sum_{k=1}^{N} P(k) \cdot \text{rel}(k), \tag{11}$$

where $P(k)$ refers to the Precision metric calculated to the first k elements of the recommendation list and $\text{rel}(k)$ is the operation that verifies if the item that is in the recommendation list in the position k is the same that is in the position k in the testing list, returning 1 if it is true or 0 if it is false.

The NDCG metric, in contrast to the MAP metric, does not favor the items that appear first in the recommendation list. Its goal, as defined by [19], is to offer a metric that is appropriated to large recommendation lists in which the penalty is applied to the items that are further from the beginning of the list. Assuming that a user u has a gain $g_{u,i}$ for being recommended an item i to it, the mean of the metric Discounted Cumulative Gain (DCG) for a list of J items can be defined as:

$$\text{DCG} = \frac{1}{N} \sum_{u=1}^{N} \sum_{j=1}^{J} \frac{g_{u,i_j}}{\log_b(j+1)}, \tag{12}$$

where i_j represents the item i in the position j of the list. The base b of the logarithm can be changed, but in general the value is 2 or 10. The normalized version of the metric DCG (NDCG), can be calculated as:

$$\text{NDCG} = \frac{\text{DCG}}{\text{DCG}^*}, \tag{13}$$

where DCG^* corresponds to the DCG computed using the set of songs that the users have listened to.

Table 1. Classification of the possible result of a recommended song to a user [19].

	Song recommended	**Song not recommended**
Song listened to	true positive (tp)	false negative (fn)
Song not listened to	false positive (fp)	true negative (tn)

To optimize our model, we tested several parameters values by using Python and Keras. The best values for the Xiami dataset consisted of 256 LSTM units and embedding vector of size 256. For the Music4All dataset, we used 512 LSTM units and embedding vector of size 1024. For both datasets, the Dropout was setup to 0.2, the Activation Function adopted was *tahn*, and the Window Size was setup to 3. As we can see, although there are fewer songs in the Music4All dataset, the model needed more LSTM units and a much bigger embedding vector to capture the relevant information about songs that can be used effectively in context-aware recommendations.

Finally, to compare two context-aware recommender systems, we applied the two sided paired t-test with a 95% confidence level [16].

4.4 Results

In this subsection, we present the results for the top-5 recommendations generated by the recommender systems. As we can see in Figures 4 and 5, our proposed model was able to outperform the baseline in both datasets for all context-aware recommender systems. The metric that shows the best improvement over the baseline was MAP, with an improvement of over 25% for the CSM-TN recommender system in the Xiami dataset.

In the Music4All dataset, the contextual embedding vector obtained by our model showed small improvement in comparison to the baseline, as we can see in the metrics of the SM-TN recommender systems, which uses only the contextual embeddings. Although there was not a big improvement in the contextual embedding vector, the general embeddings exhibited great improvements over the embeddings obtained by the baseline. The M-TN recommender system, for example, showed an improvement of $6,56\%$ in F-measure and $7,14\%$ in Recall.

For the Xiami dataset, there was an improvement for both general and contextual embedding vectors in all metrics. The improvement for the CSM-UK algorithm is similar to the improvement observed in the SM-TN, which might indicates that the relationship between users are not interfering in the final results. The most significant improvement in the Xiami dataset, for the F-measure

metric, is in the CSM-TN, which combines both embeddings (contextual and general), with an improvement of $7,26\%$.

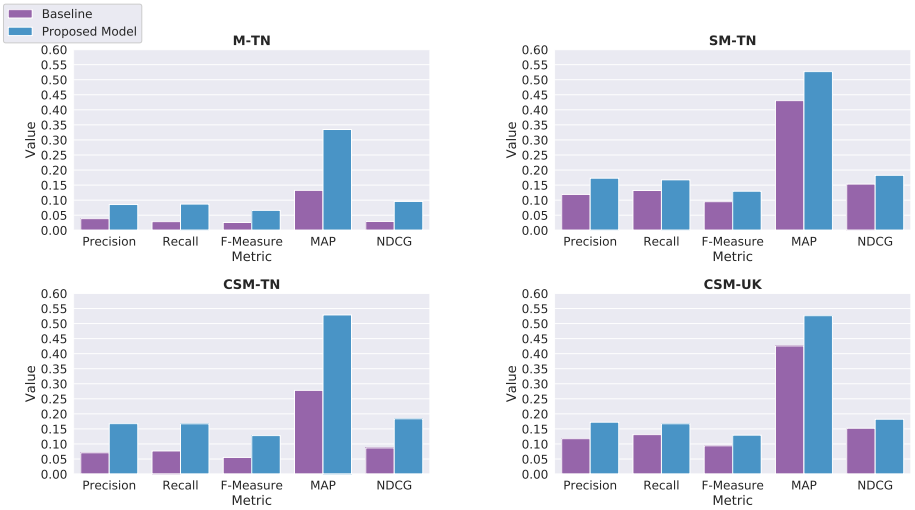

Fig. 4. Results obtained by using the embeddings from the baseline and the proposed model on the Xiami dataset.

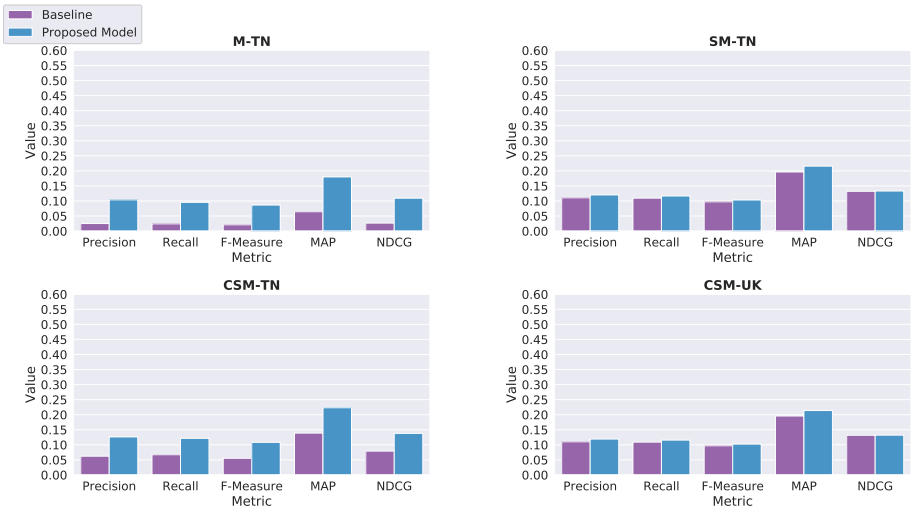

Fig. 5. Results obtained by using the embeddings from the baseline and the proposed model on the Music4All dataset.

5 Conclusion and Future Work

In this work, we proposed a model that uses LSTM units in a Dual Recurrent Neural Network to learn both general and contextual embeddings for songs that can be used in context-aware music recommender systems.

The results obtained by our model in two datasets, that contain the listening history of users, showed that our model outperformed the baseline in both datasets and for the four context-aware recommenders, using metrics that measured how good the recommendations are for the user and if the recommendations are ranked accordingly. This shows that a Recurrent Neural Network can be used effectively in capturing the intrinsic relationship between the sequence of songs that the user has listened to and generating contextual information for context-aware recommender systems.

For future work, we plan to use different Recurrent Neural Networks such as Gated Recurrent Units (GRU) [5], which has fewer parameters as LSTM and can be used to decrease the training time and memory consumption. Another possibility to improve our model is to try different methods of initializing the embedding matrices of the model.

Acknowledgments

To Conselho Nacional de Desenvolvimento Científico e Tecnológico - CNPq/Brazil (grant #403648/2016-5) for financial support and NVIDIA Corporation for donation of a Titan V GPU used in this work.

References

1. Alpaydin, E.: Design and Analysis of Machine Learning Experiments. Introduction to Machine Learning (2004)
2. Bu, J., Tan, S., Chen, C., Wang, C., Wu, H., Zhang, L., He, X.: Music recommendation by unified hypergraph. In: Proceedings of the international conference on Multimedia. p. 391. ACM Press, New York, New York, USA (2010)
3. Celma, O.: Music Recommendation. In: Music Recommendation and Discovery, pp. 43–85. Springer Berlin Heidelberg, Berlin, Heidelberg (2010)
4. Chen, H.C., Chen, A.L.P.: A Music Recommendation System Based on Music Data Grouping and User Interests. In: Proceedings of the 10th International Conference on Information and Knowledge Management. pp. 231–238. ACM, New York, NY, USA (2001)
5. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y.: Learning phrase representations using RNN encoder–decoder for statistical machine translation. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 1724–1734. Association for Computational Linguistics, Doha, Qatar (Oct 2014)
6. Chung, Y., Kim, N.R., Park, C.Y., Lee, J.H.: Improved neighborhood search for collaborative filtering. International Journal of Fuzzy Logic and Intelligent Systems **18**(1), 29–40 (mar 2018)

7. Deng, S., Wang, D., Li, X., Xu, G.: Exploring user emotion in microblogs for music recommendation. Expert Systems with Applications (2015)
8. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. The MIT Press (2016)
9. Hochreiter, S., Schmidhuber, J.: Long Short-Term Memory. Neural Computation (1997)
10. Järvelin, K., Kekäläinen, J.: Cumulated gain-based evaluation of ir techniques. ACM Trans. Inf. Syst. **20**(4), 422–446 (Oct 2002)
11. Kaminskas, M., Ricci, F.: Contextual music information retrieval and recommendation: State of the art and challenges. Computer Science Review **6**(2-3), 89–119 (5 2012)
12. Kim, Ja-Young; Belkin, N.J.: Categories of Music Description and Search Terms and Phrases Used by Non-Music Experts. In: Proceedings of the Third International Conference on Music Information Retrieval. p. 327. Paris (2002)
13. Koenigstein, N., Dror, G., Koren, Y.: Yahoo! Music Recommendations: Modeling Music Ratings with Temporal Dynamics and Item Taxonomy. In: Proceedings of the 5th ACM Conference on Recommender Systems. pp. 165–172. ACM, New York, NY, USA (2011)
14. Mayerl, M., Vötter, M., Zangerle, E., Specht, G.: Language models for next-track music recommendation. In: 31st GI-Workshop on Foundations of Databases (Grundlagen von Daten-banken) (2019)
15. Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space. In: 1st International Conference on Learning Representations, ICLR 2013 - Workshop Track Proceedings (2013)
16. Mitchell, T.M.: Machine Learning. McGraw-Hill, Inc., New York, NY, USA, 1 edn. (1997)
17. Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., Riedl, J.: GroupLens: An open architecture for collaborative filtering of netnews. In: Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work, CSCW 1994 (1994)
18. Santana, I.A.P., Pinhelli, F., Donini, J., Catharin, L., Mangolin, R.B., da Costa, Y.M.e.G., Feltrim, V.D., Domingues, M.A.: Music4All: A New Music Database and Its Applications. In: Proceedings of the 27th International Conference on Systems, Signals and Image Processing (IWSSIP 2020) (2020)
19. Shani, G., Gunawardana, A.: Evaluating recommendation systems. In: Ricci, F., Rokach, L., Shapira, B., Kantor, P.B. (eds.) Recommender Systems Handbook, chap. 8. Springer US, Boston, MA, 1 edn. (2011)
20. Vötter, M., Zangerle, E., Mayerl, M., Specht, G.: Autoencoders for next-track-recommendation. In: 31st GI-Workshop on Foundations of Databases (Grundlagen von Daten-banken) (2019)
21. Wang, D., Deng, S., Xu, G.: Sequence-based context-aware music recommendation. Information Retrieval Journal (2018)
22. Wang, D., Deng, S., Zhang, X., Xu, G.: Learning music embedding with metadata for context aware recommendation. In: ICMR 2016 - Proceedings of the 2016 ACM International Conference on Multimedia Retrieval (2016)